
Fundamentals of Scripting Language (FSL)

- Question Bank Solutions

Chapter 4: Cookies and Browser Data

1. Define cookies. What is their importance?

Cookies are small pieces of data stored in a user's web browser by a website. They store information like login details, preferences, and browsing history.

◆ Importance of Cookies:

- ✓ Maintain user sessions (e.g., keeping a user logged in).
- ✓ Store user preferences (e.g., theme settings).
- ✓ Track user behavior for analytics and advertising.
- ✓ Improve website performance and functionality.

2. Writing, Reading, and Deleting Cookies (Syntax & Example)

Writing a cookie:

```
document.cookie = "username=Sweetu; expires=Fri, 31 Dec 2025 23:59:59 UTC; path=/";
```

Reading a cookie:

```
console.log(document.cookie);
```

Deleting a cookie:

```
document.cookie = "username=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/";
```

3. Write a program to open a new window using JavaScript.

```
function openNewWindow() {  
    window.open("https://www.google.com", "_blank",  
    "width=500,height=500");  
}
```

4. Explain the syntax of the `window.open()` method.

Syntax:

```
window.open(URL, name, specs, replace);
```

Explanation:

- **URL** → The webpage to open.
 - **name** → Name of the window (`_blank` for a new tab).
 - **specs** → Settings like width, height, etc.
 - **replace** → Boolean value to replace history.
-

5. Attributes of the `window.open()` method

- 1 **width** - Sets the width of the new window.
 - 2 **height** - Sets the height of the new window.
 - 3 **left** - Defines the position from the left side of the screen.
 - 4 **top** - Defines the position from the top of the screen.
-

6. Define `open()`, `close()`, `blur()`, `focus()` methods with examples.

◆ **open()** - Opens a new window.

```
window.open("https://example.com", "_blank", "width=500,height=500");
```

◆ **close()** - Closes the current window.

```
window.close();
```

◆ **blur()** - Removes focus from the current window.

```
window.blur();
```

◆ **focus()** - Brings the window back into focus.

```
window.focus();
```

7. Define `setTimeout()` and `setInterval()` with examples.

◆ **setTimeout()** - Executes a function **after a delay**.

```
setTimeout(() => {  
    console.log("This appears after 3 seconds!");  
}, 3000);
```

◆ **setInterval()** - Executes a function **repeatedly**.

```
setInterval(() => {
```

```
    console.log("This appears every 2 seconds!");  
  }, 2000);
```

8. Explain `confirm()` method with example.

Displays a dialog box with **OK** and **Cancel** options.

```
if (confirm("Do you want to proceed?")) {  
    console.log("User clicked OK");  
} else {  
    console.log("User clicked Cancel");  
}
```

9. Explain `prompt()` method with example.

Displays a dialog box that allows user input.

```
let name = prompt("Enter your name:");  
console.log("Hello, " + name);
```

Chapter 5: Regular Expressions and Rollover Effects

1. Write a JavaScript program to send content from a parent window to a child window.

```
let childWin;  
function openChildWindow() {  
    childWin = window.open("", "childWindow", "width=400,height=400");  
}  
function writeToChild() {  
    if (childWin) {  
        childWin.document.write("<p>Text added from parent!</p>");  
    }  
}
```

2. Give examples of Regular Expressions in JavaScript.

```
let email = "user@example.com";  
console.log(/^\w+@\w+\.\w+$/ .test(email)); // Validates email  
  
let phone = "9876543210";  
console.log(/^\d{10}$/ .test(phone)); // Validates 10-digit phone number
```

3. Write a script to change the content of a child window dynamically.

```
let childWin;  
function openChildWindow() {  
    childWin = window.open("", "childWindow", "width=400,height=400");
```

```
    childWin.document.write("<h1>Hello from Parent!</h1>");
}
function changeChildContent() {
    if (childWin) {
        childWin.document.body.innerHTML = "<h2>Content Updated!</h2>";
    }
}
```

4. Write a JavaScript program to create a rollover effect (change text color on hover).

```
<p id="rolloverText" onmouseover="changeColor()"
onmouseout="resetColor()">Hover over me!</p>

<script>
function changeColor() {
    document.getElementById("rolloverText").style.color = "red";
}
function resetColor() {
    document.getElementById("rolloverText").style.color = "black";
}
</script>
```

5. Write a script for a rollover effect using images.

```

<p onmouseover="showBook()" onmouseout="hideBook()">Hover over this
text</p>

<script>
function showBook() {
    document.getElementById("bookImage").src = "book.jpg";
}
function hideBook() {
    document.getElementById("bookImage").src = "default.jpg";
}
</script>
```

6. Define pattern matching using regular expressions.

Pattern matching is a technique used in **regular expressions (Regex)** to search, match, and manipulate text based on patterns. It allows for operations like validation, replacement, and searching within strings.

◆ Example: Matching an email pattern

```
let email = "hello@example.com";
let emailPattern = /^\\w+@\\w+\\.\\w+$/;
console.log(emailPattern.test(email)); // Output: true
```

7. Explain predefined character classes in regular expressions.

Predefined character classes are shortcuts used in **RegEx** to match specific types of characters.

Character Class	Meaning	Example
<code>\d</code>	Matches any digit (0-9)	<code>/\d+/</code> → Matches "123" in "abc123xyz"
<code>\w</code>	Matches any word character (a-z, A-Z, 0-9, _)	<code>/\w+/</code> → Matches "hello_123"
<code>\s</code>	Matches any whitespace (spaces, tabs, newlines)	<code>/\s+/</code> → Matches " " in "Hello World"
<code>\D</code>	Matches any non-digit	<code>/\D+/</code> → Matches "abc" in "abc123"
<code>\W</code>	Matches any non-word character	<code>/\W+/</code> → Matches "@" in "hello@"
<code>\S</code>	Matches any non-whitespace character	<code>/\S+/</code> → Matches "Hello" in "Hello World"

◆ Example:

```
let text = "My number is 9876543210";  
console.log(text.match(/\d+/)); // Output: ["9876543210"]
```

8. Explain quantifiers in regular expressions with examples.

Quantifiers define **how many times a character, group, or class should appear** in a string.

Quantifier	Meaning	Example
<code>+</code>	One or more occurrences	<code>/\d+/</code> → Matches "123" in "abc123xyz"
<code>*</code>	Zero or more occurrences	<code>/a*/</code> → Matches "aaa" in "aaaabc"
<code>?</code>	Zero or one occurrence	<code>/colou?r/</code> → Matches "color" & "colour"
<code>{n}</code>	Exactly n occurrences	<code>/\d{4}/</code> → Matches "2024" in "year2024"
<code>{n,}</code>	At least n occurrences	<code>/\d{2,}/</code> → Matches "1234" in "abc1234xyz"
<code>{n,m}</code>	Between n and m occurrences	<code>/\d{2,4}/</code> → Matches "123" or "1234"

◆ Example:

```
let pattern = /\d{2,4}/;  
console.log("The year is 2024".match(pattern)); // Output: ["2024"]
```

9. Explain the `test()` and `exec()` methods in JavaScript regex.

◆ `test()` Method

- Checks **if a pattern exists** in a string.
- Returns **true** or **false**.

```
let regex = /hello/;
console.log(regex.test("hello world")); // Output: true
console.log(regex.test("goodbye")); // Output: false
```

◆ **exec()** Method

- Searches for a match and returns the **first occurrence**.
- Returns **null** if no match is found.

```
let text = "The number is 9876543210";
let pattern = /\d+/;
console.log(pattern.exec(text)); // Output: ["9876543210"]
```

Chapter 6: Navigation and JavaScript Frameworks (Complete Version)

1. Write a script to create rotating banner ads.

```


<script>
let images = ["ad1.jpg", "ad2.jpg", "ad3.jpg"];
let index = 0;

function rotateBanner() {
    index = (index + 1) % images.length;
    document.getElementById("banner").src = images[index];
}

setInterval(rotateBanner, 3000);
</script>
```

2. Write a script to create a slideshow.

```


<script>
let slides = ["slide1.jpg", "slide2.jpg", "slide3.jpg"];
let count = 0;

function nextSlide() {
    count = (count + 1) % slides.length;
    document.getElementById("slideshow").src = slides[count];
}

setInterval(nextSlide, 2000);
</script>
```

3. Explain types of JavaScript frameworks.

✓ Frontend Frameworks:

- **React.js** – UI component-based framework.
- **Angular.js** – Google-backed MVC framework.
- **Vue.js** – Lightweight, flexible UI framework.

✓ Backend Frameworks:

- **Node.js** – JavaScript runtime for server-side scripting.
 - **Express.js** – Minimalist web framework for Node.js.
 - **Meteor.js** – Full-stack framework for real-time applications.
-

4. Explain the advantages of JavaScript frameworks.

✓ **1. Faster Development** – Frameworks provide pre-written code, reducing development time.

✓ **2. Code Reusability** – Components can be reused across multiple projects.

✓ **3. Scalability** – Helps in building large-scale applications efficiently.

✓ **4. Cross-Browser Compatibility** – Ensures consistent behavior across different web browsers.

✓ **5. Security Features** – Built-in security mechanisms to protect against vulnerabilities.

✓ **6. Community Support** – Active developer communities provide regular updates and support.
